

Validity

What the reward-tilted proposal does to parseability, and whether screening for it helps

2026-08-19

1 Why validity is the interesting axis

Our reward is evaluated on a *clean* x_0 candidate and returns `invalid_reward` = 0 when RDKit cannot parse it. An unparseable candidate is therefore treated as a property-zero one and suppressed by the tilt: raising λ buys *parseability* before it buys property.

D-CBG has no such term. It perturbs the per-position logits with the gradient of a noisy classifier, and nothing in that gradient prefers a parseable string.

This single difference explains two things reported elsewhere. It is why the novel *count* rises with λ for us and falls with γ for D-CBG — novel molecules are counted as a subset of valid ones, so whatever happens to validity happens to the novel count. And it is why our curve in (novel, property) space runs up and to the right while D-CBG’s runs up and to the left.

One caveat, stated up front: the effect is not symmetric across base models, and the naive claim “our method keeps validity high” is false. See Section 3.

2 What each dial does to validity

model	method	weakest dial	best	strongest dial	trend
MDLM	ours ($N = 2000$)	544	750	750	rises
MDLM	D-CBG exact	537	537	33	collapses
MDLM	D-CBG approx.	504	504	108	collapses
UDLM	ours ($N = 2000$)	694	937	903	rises
UDLM	D-CBG exact	949	989	989	rises
UDLM	D-CBG approx.	997	997	819	falls

Table 1: QED: valid molecules out of 1,024 as the strength dial is turned up. Ours along λ at the largest N ; D-CBG along γ .

3 The asymmetry

On **MDLM** the contrast is as stark as the mechanism predicts: ours climbs with λ while D-CBG’s exact arm collapses, reaching *zero* valid molecules at $\gamma \geq 6$ — which is why the original paper does

model	method	weakest dial	best	strongest dial	trend
MDLM	ours ($N = 2000$)	544	734	734	rises
MDLM	D-CBG exact	188	188	2	collapses
MDLM	D-CBG approx.	493	493	88	collapses
UDLM	ours ($N = 2000$)	694	974	974	rises
UDLM	D-CBG exact	807	910	892	rises
UDLM	D-CBG approx.	992	992	792	falls

Table 2: ring count: valid molecules out of 1,024 as the strength dial is turned up. Ours along λ at the largest N ; D-CBG along γ .

not report that region. The collapse is not cosmetic: it caps how far D-CBG can push the property at all on this base model.

On **UDLM** the claim fails. D-CBG’s exact arm *raises* validity as γ grows and is above ours at every setting, while ours peaks at moderate λ and then declines slightly. A plausible reason is that uniform diffusion resamples every position at every step and so can repair a local mistake later, whereas absorbing-state diffusion cannot revisit an unmasked token — but that is a hypothesis this report does not test.

So the defensible claim is narrower and about robustness rather than level:

Our dial does not destroy validity. It has no collapse regime, so the property can be pushed as far as compute allows. D-CBG on MDLM has one, and it bounds what that method can reach.

4 Does screening candidates for validity help?

If the tilt spends its early range on validity, that range is being consumed by something a cheap filter could do instead. Three arms separate the two effects, all on QED with a single seed per cell:

A `oversample= 1`, unrestricted target. The method as every earlier result ran it: draw N candidates, score all of them, and give an unparseable one the reward `invalid_reward = 0`. Run at $\lambda = 5000$ only.

A’ the earlier $\lambda \times N$ sweep, the one behind `qed_sweep.pdf`, reused here to fill the cells A never ran — and it is exactly complementary, since that sweep covered $\lambda \leq 1000$ and A covers only $\lambda = 5000$. **It is not arm A.** Those runs predate `exact_uniform_step`, so wherever the importance weights come out uniform, A’ takes a Monte Carlo step where A substitutes the closed-form base kernel.

The trigger is a *reward tie* — `spread` $< 10^{-9}$ on the weights — and **not** a validity flag, which is why it fires in arm A even though arm A never masks on validity. Validity still reaches it, because `invalid_reward = 0` is what an unparseable candidate scores in *every* arm; masking the logits (`exclude_invalid`) is a separate mechanism. Two causes, measured at $\lambda = 5000$, $N = 300$ as medians over the trajectory: every candidate unparseable (MDLM 0.266, UDLM 0.156), and ties among parseable candidates — duplicates decoding to the same molecule, or distinct molecules of equal QED — which add 0.047 on MDLM (up to 0.719) and essentially

nothing on UDLM. Total uniform rate: median 0.359 on MDLM against 0.156 on UDLM. The duplicate half is the same effect behind the ESS floor, and it is an MDLM phenomenon. So A' is reported as its own column and never averaged with A .

B `oversample= 1`, *valid-restricted* target. Same candidates, but $q(x_0) = 0$ wherever RDKit cannot read x_0 , so invalid candidates carry zero weight. Isolates restricting the target.

C `oversample= 5`, *valid-restricted and screened*. Draw $5N$, keep N preferring the parseable ones, score those. Isolates improving the proposal.

B versus C is the effect of screening, at the same target and the same N . A versus B is the effect of restricting the target at all.

Why this is correct rather than a heuristic: restricting to valid molecules makes the proposal $p_\theta(\cdot \mid \text{valid})$, and the resulting normaliser $Z = P_\theta(\text{valid} \mid x_t)$ does not depend on x_0 , so it cancels under self-normalisation and Eq. (8) is unchanged. Conditioning i.i.d. draws on an event gives i.i.d. draws from the conditional, and a uniformly random subset of them preserves that — so C is a valid estimator for the same target as B , reached with N usable candidates instead of $N \times (\text{valid fraction})$, a fraction measured as low as 0.007 early in a UDLM trajectory.

The cost argument is measured, not assumed: parsing costs 0.042ms against 0.811ms for parse-plus-QED, so the reward is 91% of a full evaluation. Screening therefore leaves the expensive operation almost untouched. Note the budgets are matched on N , not on cost: C scores more *valid* candidates than B , so it makes more reward calls. A cost-matched comparison is available post hoc against the $\lambda \times N$ sweep, which runs `over1` up to $N = 2000$.

λ starts at 20 rather than 0. At $\lambda = 0$ there is no importance weighting to reduce the variance of, so the claim this experiment tests does not apply there.

Arms present in this build: **A** `over1`, unrestricted target — the method as every earlier result ran it, **A_{eu}** as A , plus the exact-uniform step ($\lambda = 5000$ only; an ablation, not the baseline), **B** `over1`, valid-restricted, **C** `over5`, valid-restricted + screening.

5 Is screening worth its cost?

Every contrast so far pairs at fixed N , and N is not the quantity anyone spends — time is. Screening draws $5N$ candidates and has to decode and parse all of them, so the arms are not cost-matched at equal N . Measured slowdown of C over B : 1.26, 1.66, 4.71, 7.50 on MDLM and 1.23, 2.25, 4.82, 6.28 on UDLM at $N = 10, 30, 100, 300$.

The verdict changes. At matched *cost* the advantage largely goes away. B at $N = 300$ costs 121s on MDLM and reaches 221/115/37/7/1 hits across the five bars; C at $N = 100$ costs 320s — $2.6\times$ as much — and reaches 208/114/42/10/0. It is *worse* at the 0.50 bar, tied at 0.55, better at 0.60 and 0.65. UDLM tells the same story: B at $N = 300$ (143s) gives 266/156/67/14/3 against C at $N = 100$ (371s) giving 233/154/74/16/0. **So the paired $C-B$ table overstates the benefit**, because holding N fixed hides that screening bought its extra valid candidates with time that could have been spent on more candidates instead.

arm	N	valid	novel	novel-mean	valid	novel	novel-mean	best λ
A	10	638	255	0.4772	648	268	0.4816	1000
A	30	664	289	0.4869	675	304	0.4933	200
A	100	692	313	0.4952	727	336	0.5037	5000
A	300	714	350	0.4994	723	396	0.5127	200
A _{eu}	10	628	263	0.4801	642	273	0.4813	5000
A _{eu}	30	658	286	0.4890	666	288	0.4907	5000
A _{eu}	100	669	334	0.4987	680	338	0.5007	5000
A _{eu}	300	714	392	0.5070	714	393	0.5118	5000
B	10	628	260	0.4769	643	273	0.4815	5000
B	30	657	284	0.4844	669	292	0.4913	200
B	100	670	323	0.4908	686	338	0.5007	1000
B	300	707	360	0.4990	714	393	0.5118	1000
C	10	643	278	0.4809	661	292	0.4899	1000
C	30	668	309	0.4977	681	330	0.5077	5000
C	100	702	346	0.5028	711	381	0.5154	5000
C	300	742	403	0.5066	767	452	0.5194	1000

mean over cells | **max** over cells. *valid* and *novel* are counts out of 1,024 generated; *novel-mean* is the mean property over the novel molecules, i.e. the metric the paper reports. Note that novel-mean and the novel count trade off against each other, so neither alone ranks the arms.

Table 3: MDLM: the three quantities behind hits. First block is the mean over the $\lambda \times$ mode cells at that (arm, N), second block the best single cell. *best* λ is the one maximising hits at the QED ≥ 0.6 bar.

arm	N	valid	novel	novel-mean	valid	novel	novel-mean	best λ
A	10	891	234	0.4998	900	240	0.5121	200
A	30	924	254	0.5118	936	272	0.5246	5000
A	100	927	287	0.5196	952	307	0.5348	5000
A	300	936	302	0.5247	963	327	0.5426	1000
A _{eu}	10	883	249	0.5044	888	250	0.5111	5000
A _{eu}	30	924	266	0.5209	932	271	0.5243	5000
A _{eu}	100	922	284	0.5314	926	308	0.5341	5000
A _{eu}	300	922	320	0.5381	934	336	0.5384	5000
B	10	882	246	0.4989	894	253	0.5111	5000
B	30	925	269	0.5105	943	283	0.5243	200
B	100	928	278	0.5197	960	308	0.5341	5000
B	300	930	301	0.5252	969	347	0.5414	1000
C	10	921	230	0.5009	935	250	0.5144	1000
C	30	933	257	0.5202	954	287	0.5373	200
C	100	943	294	0.5272	972	327	0.5448	1000
C	300	940	327	0.5337	978	369	0.5538	1000

mean over cells | **max** over cells. *valid* and *novel* are counts out of 1,024 generated; *novel-mean* is the mean property over the novel molecules, i.e. the metric the paper reports. Note that novel-mean and the novel count trade off against each other, so neither alone ranks the arms.

Table 4: UDLM: the three quantities behind hits. First block is the mean over the $\lambda \times$ mode cells at that (arm, N), second block the best single cell. *best* λ is the one maximising hits at the QED ≥ 0.6 bar.

arm	N	marg. 20	marg. 50	marg. 200	marg. 1000	marg. 5000	edlm. 20	edlm. 50	edlm. 200	edlm. 1000	edlm. 5000
A	10	0.4670	0.4783	0.4795	0.4816	0.4802	0.4730	0.4744	0.4787	0.4798	0.4798
A	30	0.4699	0.4871	0.4917	0.4905	0.4908	0.4780	0.4870	0.4890	0.4919	0.4933
A	100	0.4779	0.4935	0.4946	0.4993	0.5004	0.4830	0.4953	0.5010	0.5030	0.5037
A	300	0.4780	0.4971	0.5090	0.5124	0.5127	0.4813	0.4946	0.5010	0.5038	0.5044
A _{eu}	10	—	—	—	—	0.4813	—	—	—	—	0.4790
A _{eu}	30	—	—	—	—	0.4873	—	—	—	—	0.4907
A _{eu}	100	—	—	—	—	0.4968	—	—	—	—	0.5007
A _{eu}	300	—	—	—	—	0.5118	—	—	—	—	0.5021
B	10	0.4730	0.4771	0.4815	0.4805	0.4813	0.4658	0.4734	0.4783	0.4792	0.4790
B	30	0.4699	0.4817	0.4860	0.4861	0.4873	0.4749	0.4851	0.4912	0.4913	0.4907
B	100	0.4719	0.4855	0.4958	0.4972	0.4968	0.4764	0.4859	0.4992	0.4988	0.5007
B	300	0.4807	0.4975	0.5091	0.5109	0.5118	0.4839	0.4931	0.4996	0.5008	0.5021
C	10	0.4678	0.4755	0.4816	0.4832	0.4834	0.4738	0.4805	0.4841	0.4899	0.4894
C	30	0.4850	0.4944	0.5018	0.5007	0.5002	0.4787	0.4965	0.5049	0.5067	0.5077
C	100	0.4828	0.4964	0.5065	0.5078	0.5086	0.4845	0.4997	0.5125	0.5154	0.5136
C	300	0.4779	0.4970	0.5160	0.5194	0.5186	0.4858	0.5031	0.5151	0.5168	0.5164

Every cell is a single run of 1,024 sequences; nothing is averaged. Arm A_{eu} ran at $\lambda = 5000$ only. The earlier sweep behind arm A also covers $N \in \{500, 1000, 2000\}$ and $\lambda \in \{0, 2, 5\}$, which are outside this grid and are reported in [qed_sweep.pdf](#); the same numbers are exported to [pdfs/Validity/notes/quality_by_lambda.csv](#) for plotting.

Table 5: MDLM: QED novel-mean per run, resolved by λ . Columns are (mixture mode) $\times \lambda$; rows group by arm. Read one arm block as a plot: $x = N$, one series per λ , **marginal** against **edlm**.

arm	N	marg. 20	marg. 50	marg. 200	marg. 1000	marg. 5000	edlm. 20	edlm. 50	edlm. 200	edlm. 1000	edlm. 5000
A	10	0.4760	0.4923	0.5056	0.5023	0.5061	0.4818	0.5014	0.5121	0.5104	0.5102
A	30	0.4872	0.5050	0.5152	0.5224	0.5219	0.4882	0.5076	0.5211	0.5246	0.5244
A	100	0.4892	0.5064	0.5212	0.5286	0.5275	0.5019	0.5194	0.5330	0.5348	0.5337
A	300	0.4854	0.5131	0.5330	0.5333	0.5341	0.4937	0.5297	0.5426	0.5418	0.5408
A _{eu}	10	—	—	—	—	0.4977	—	—	—	—	0.5111
A _{eu}	30	—	—	—	—	0.5176	—	—	—	—	0.5243
A _{eu}	100	—	—	—	—	0.5287	—	—	—	—	0.5341
A _{eu}	300	—	—	—	—	0.5378	—	—	—	—	0.5384
B	10	0.4798	0.4893	0.4980	0.4982	0.4977	0.4886	0.5052	0.5104	0.5106	0.5111
B	30	0.4840	0.4987	0.5149	0.5172	0.5176	0.4932	0.5115	0.5214	0.5221	0.5243
B	100	0.4834	0.5091	0.5244	0.5284	0.5287	0.5001	0.5218	0.5328	0.5337	0.5341
B	300	0.4928	0.5100	0.5325	0.5384	0.5378	0.4919	0.5292	0.5394	0.5414	0.5384
C	10	0.4644	0.4968	0.5063	0.5144	0.5124	0.4869	0.5024	0.5079	0.5089	0.5091
C	30	0.4826	0.5005	0.5364	0.5360	0.5373	0.4917	0.5168	0.5367	0.5337	0.5307
C	100	0.4940	0.5123	0.5299	0.5415	0.5448	0.5071	0.5236	0.5392	0.5411	0.5385
C	300	0.4896	0.5176	0.5364	0.5490	0.5479	0.5078	0.5309	0.5516	0.5527	0.5538

Every cell is a single run of 1,024 sequences; nothing is averaged. Arm A_{eu} ran at $\lambda = 5000$ only. The earlier sweep behind arm A also covers $N \in \{500, 1000, 2000\}$ and $\lambda \in \{0, 2, 5\}$, which are outside this grid and are reported in [qed_sweep.pdf](#); the same numbers are exported to [pdfs/Validity/notes/quality_by_lambda.csv](#) for plotting.

Table 6: UDLM: QED novel-mean per run, resolved by λ . Columns are (mixture mode) $\times \lambda$; rows group by arm. Read one arm block as a plot: $x = N$, one series per λ , **marginal** against **edlm**.

N	λ	marg. A	marg. A _{eu}	marg. B	marg. C	edlm. A	edlm. A _{eu}	edlm. B	edlm. C
10	20	74	–	86	85	79	–	84	84
10	50	92	–	88	98	87	–	89	104
10	200	92	–	99	105	92	–	105	108
10	1000	99	–	99	106	92	–	104	116
10	5000	96	101	101	107	93	104	104	113
30	20	90	–	99	121	88	–	93	99
30	50	113	–	114	137	108	–	107	128
30	200	132	–	124	167	122	–	115	167
30	1000	127	–	122	163	130	–	118	175
30	5000	127	121	121	161	135	118	118	172
100	20	92	–	92	119	126	–	109	129
100	50	117	–	106	147	149	–	134	139
100	200	133	–	140	180	168	–	162	192
100	1000	146	–	148	200	174	–	165	208
100	5000	148	146	146	205	175	168	168	202
300	20	109	–	117	119	102	–	122	154
300	50	150	–	151	160	145	–	155	207
300	200	208	–	207	224	177	–	177	249
300	1000	215	–	211	254	179	–	187	261
300	5000	220	221	221	257	180	194	194	264

Table 7: MDLM: hits@0.5 per run. Columns are (mixture mode) $\times$ (arm); every cell is a single run of 1,024 sequences, with no averaging.

N	λ	marg. A	marg. A _{eu}	marg. B	marg. C	edlm. A	edlm. A _{eu}	edlm. B	edlm. C
10	20	101	–	107	85	91	–	113	124
10	50	111	–	119	114	125	–	131	137
10	200	128	–	134	123	144	–	154	138
10	1000	118	–	138	145	144	–	151	143
10	5000	123	139	139	143	144	152	152	148
30	20	106	–	117	131	103	–	125	137
30	50	124	–	141	130	139	–	165	167
30	200	160	–	173	162	176	–	193	184
30	1000	179	–	171	181	187	–	181	178
30	5000	179	167	167	182	185	190	190	171
100	20	138	–	121	134	165	–	153	155
100	50	152	–	130	151	180	–	198	171
100	200	186	–	170	197	230	–	219	208
100	1000	207	–	189	227	220	–	224	231
100	5000	206	185	185	233	221	223	223	226
300	20	103	–	129	127	157	–	134	172
300	50	160	–	146	169	198	–	201	215
300	200	229	–	219	229	243	–	249	288
300	1000	232	–	238	276	255	–	266	293
300	5000	230	235	235	270	252	252	252	292

Table 8: UDLM: hits@0.5 per run. Columns are (mixture mode) $\times$ (arm); every cell is a single run of 1,024 sequences, with no averaging.

N	λ	marg. A	marg. A _{eu}	marg. B	marg. C	edlm. A	edlm. A _{eu}	edlm. B	edlm. C
10	20	28	–	32	31	24	–	27	40
10	50	37	–	32	33	31	–	31	48
10	200	37	–	32	38	36	–	41	50
10	1000	38	–	32	43	36	–	44	54
10	5000	38	33	33	43	36	44	44	53
30	20	35	–	34	58	38	–	32	45
30	50	51	–	43	73	50	–	47	61
30	200	60	–	47	83	57	–	55	72
30	1000	57	–	48	78	59	–	57	78
30	5000	56	49	49	76	61	57	57	78
100	20	38	–	32	49	49	–	48	58
100	50	53	–	49	67	63	–	58	77
100	200	60	–	75	85	73	–	73	99
100	1000	69	–	80	97	81	–	74	114
100	5000	72	77	77	98	80	77	77	110
300	20	45	–	52	54	50	–	54	63
300	50	64	–	68	78	66	–	77	102
300	200	98	–	99	117	83	–	89	132
300	1000	111	–	115	132	89	–	97	135
300	5000	107	114	114	130	89	99	99	135

Table 9: MDLM: hits@0.55 per run. Columns are (mixture mode) $\times$ (arm); every cell is a single run of 1,024 sequences, with no averaging.

N	λ	marg. A	marg. A _{eu}	marg. B	marg. C	edlm. A	edlm. A _{eu}	edlm. B	edlm. C
10	20	39	–	41	35	44	–	46	50
10	50	45	–	46	63	61	–	70	65
10	200	63	–	60	64	67	–	74	63
10	1000	55	–	66	76	62	–	78	64
10	5000	57	65	65	70	63	78	78	65
30	20	55	–	48	53	51	–	57	67
30	50	69	–	57	61	74	–	90	86
30	200	81	–	83	100	87	–	100	112
30	1000	99	–	89	108	99	–	102	105
30	5000	95	88	88	115	93	104	104	100
100	20	54	–	49	68	74	–	77	78
100	50	63	–	57	91	91	–	104	91
100	200	96	–	99	115	145	–	124	132
100	1000	105	–	105	139	130	–	128	154
100	5000	106	100	100	141	130	131	131	150
300	20	50	–	54	50	69	–	50	87
300	50	87	–	83	98	109	–	107	115
300	200	132	–	123	153	141	–	143	185
300	1000	126	–	131	191	150	–	156	200
300	5000	127	129	129	174	149	146	146	202

Table 10: UDLM: hits@0.55 per run. Columns are (mixture mode) $\times$ (arm); every cell is a single run of 1,024 sequences, with no averaging.

N	λ	marg. A	marg. A _{eu}	marg. B	marg. C	edlm. A	edlm. A _{eu}	edlm. B	edlm. C
10	20	7	–	7	9	9	–	10	11
10	50	11	–	9	12	10	–	9	13
10	200	12	–	9	11	10	–	12	15
10	1000	13	–	9	12	11	–	13	19
10	5000	12	10	10	13	11	13	13	17
30	20	8	–	7	19	14	–	9	18
30	50	12	–	8	26	15	–	14	23
30	200	19	–	13	33	17	–	16	28
30	1000	18	–	14	34	18	–	15	36
30	5000	18	14	14	34	18	15	15	36
100	20	10	–	12	14	13	–	7	14
100	50	20	–	18	24	19	–	23	30
100	200	22	–	26	34	24	–	25	31
100	1000	29	–	30	39	31	–	27	27
100	5000	28	28	28	42	34	30	30	27
300	20	12	–	9	12	9	–	15	18
300	50	23	–	22	28	23	–	18	29
300	200	39	–	35	50	27	–	26	42
300	1000	39	–	37	55	33	–	29	37
300	5000	38	37	37	52	31	28	28	40

Table 11: MDLM: hits@0.6 per run. Columns are (mixture mode) $\times$ (arm); every cell is a single run of 1,024 sequences, with no averaging.

N	λ	marg. A	marg. A _{eu}	marg. B	marg. C	edlm. A	edlm. A _{eu}	edlm. B	edlm. C
10	20	9	–	12	6	14	–	9	13
10	50	13	–	9	18	15	–	11	14
10	200	15	–	13	19	24	–	10	16
10	1000	11	–	14	21	23	–	13	18
10	5000	14	15	15	21	22	13	13	20
30	20	12	–	12	14	18	–	13	16
30	50	27	–	17	20	18	–	27	24
30	200	17	–	27	37	17	–	31	42
30	1000	27	–	28	36	27	–	27	41
30	5000	29	28	28	38	25	28	28	38
100	20	17	–	16	18	15	–	17	22
100	50	20	–	23	33	31	–	33	35
100	200	35	–	36	47	38	–	43	56
100	1000	31	–	31	74	42	–	46	62
100	5000	30	31	31	72	42	49	49	67
300	20	9	–	14	11	8	–	12	20
300	50	21	–	21	36	34	–	28	42
300	200	43	–	42	56	55	–	57	90
300	1000	47	–	54	78	59	–	67	98
300	5000	50	52	52	75	58	57	57	91

Table 12: UDLM: hits@0.6 per run. Columns are (mixture mode) $\times$ (arm); every cell is a single run of 1,024 sequences, with no averaging.

N	λ	marg. A	marg. A _{eu}	marg. B	marg. C	edlm. A	edlm. A _{eu}	edlm. B	edlm. C
10	20	0	—	0	0	0	—	1	1
10	50	2	—	0	2	0	—	0	0
10	200	2	—	0	2	0	—	1	1
10	1000	3	—	0	1	0	—	1	2
10	5000	3	0	0	1	0	1	1	2
30	20	2	—	2	2	0	—	1	1
30	50	2	—	2	2	1	—	1	4
30	200	4	—	3	6	2	—	1	3
30	1000	4	—	3	7	1	—	1	4
30	5000	4	3	3	6	1	1	1	4
100	20	2	—	3	2	0	—	1	2
100	50	4	—	4	4	1	—	4	5
100	200	1	—	4	7	3	—	5	5
100	1000	2	—	4	9	3	—	6	7
100	5000	2	4	4	10	4	6	6	6
300	20	0	—	0	1	2	—	3	4
300	50	4	—	4	1	2	—	1	3
300	200	4	—	3	8	4	—	5	5
300	1000	4	—	3	9	4	—	7	9
300	5000	4	3	3	9	5	7	7	8

Table 13: MDLM: hits@0.65 per run. Columns are (mixture mode) $\times$ (arm); every cell is a single run of 1,024 sequences, with no averaging.

N	λ	marg. A	marg. A _{eu}	marg. B	marg. C	edlm. A	edlm. A _{eu}	edlm. B	edlm. C
10	20	0	—	1	0	2	—	1	1
10	50	0	—	1	1	1	—	2	0
10	200	2	—	2	2	4	—	0	0
10	1000	1	—	2	1	4	—	0	1
10	5000	2	2	2	1	4	0	0	0
30	20	2	—	1	1	3	—	2	1
30	50	3	—	4	1	1	—	5	2
30	200	2	—	2	2	2	—	6	6
30	1000	5	—	1	3	3	—	6	8
30	5000	5	1	1	3	3	5	5	6
100	20	1	—	1	2	2	—	1	2
100	50	2	—	4	5	1	—	2	5
100	200	11	—	10	6	4	—	5	8
100	1000	6	—	8	16	6	—	5	10
100	5000	6	6	6	15	4	5	5	11
300	20	1	—	2	1	1	—	0	1
300	50	2	—	3	3	5	—	3	5
300	200	6	—	7	9	7	—	14	18
300	1000	10	—	14	9	9	—	13	24
300	5000	11	12	12	9	8	12	12	20

Table 14: UDLN: hits@0.65 per run. Columns are (mixture mode) $\times$ (arm); every cell is a single run of 1,024 sequences, with no averaging.

N	λ	marg. A	marg. A _{eu}	marg. B	marg. C	edlm. A	edlm. A _{eu}	edlm. B	edlm. C
10	20	0	–	0	0	0	–	0	0
10	50	0	–	0	0	0	–	0	0
10	200	0	–	0	0	0	–	0	0
10	1000	0	–	0	0	0	–	0	0
10	5000	0	0	0	0	0	0	0	0
30	20	0	–	0	0	0	–	0	0
30	50	0	–	0	0	0	–	0	0
30	200	0	–	0	0	0	–	0	0
30	1000	0	–	0	0	0	–	0	0
30	5000	0	0	0	0	0	0	0	0
100	20	0	–	0	0	0	–	0	0
100	50	0	–	0	0	0	–	0	0
100	200	0	–	0	0	0	–	0	0
100	1000	0	–	0	0	0	–	0	0
100	5000	0	0	0	0	0	0	0	0
300	20	0	–	0	0	0	–	0	0
300	50	0	–	0	0	0	–	0	0
300	200	0	–	0	0	0	–	0	0
300	1000	0	–	0	1	0	–	1	0
300	5000	0	0	0	0	0	1	1	0

Table 15: MDLM: hits@0.7 per run. Columns are (mixture mode) $\times$ (arm); every cell is a single run of 1,024 sequences, with no averaging.

N	λ	marg. A	marg. A _{eu}	marg. B	marg. C	edlm. A	edlm. A _{eu}	edlm. B	edlm. C
10	20	0	–	0	0	0	–	0	0
10	50	0	–	0	0	0	–	0	0
10	200	0	–	0	0	0	–	0	0
10	1000	0	–	0	0	0	–	0	0
10	5000	0	0	0	0	0	0	0	0
30	20	0	–	0	0	0	–	0	0
30	50	0	–	0	0	0	–	0	0
30	200	0	–	0	0	0	–	0	1
30	1000	0	–	0	0	0	–	0	1
30	5000	0	0	0	0	0	0	0	1
100	20	0	–	0	0	0	–	0	0
100	50	0	–	0	0	0	–	0	0
100	200	0	–	0	0	0	–	0	0
100	1000	0	–	0	0	0	–	0	0
100	5000	0	0	0	0	0	0	0	0
300	20	0	–	0	0	0	–	0	0
300	50	0	–	0	0	0	–	0	2
300	200	1	–	1	0	1	–	1	2
300	1000	1	–	2	0	0	–	0	0
300	5000	2	3	3	0	0	1	1	0

Table 16: UDLM: hits@0.7 per run. Columns are (mixture mode) $\times$ (arm); every cell is a single run of 1,024 sequences, with no averaging.

N	λ	marg. A	marg. A _{eu}	marg. B	marg. C	edlm. A	edlm. A _{eu}	edlm. B	edlm. C
10	20	638	–	619	627	634	–	643	661
10	50	646	–	615	628	631	–	642	659
10	200	647	–	613	634	630	–	641	648
10	1000	648	–	614	637	630	–	641	651
10	5000	648	615	615	634	632	642	642	652
30	20	650	–	660	676	670	–	642	657
30	50	650	–	667	669	670	–	650	657
30	200	655	–	669	671	673	–	648	665
30	1000	659	–	668	681	675	–	650	667
30	5000	658	666	666	678	675	650	650	664
100	20	662	–	653	697	725	–	685	697
100	50	665	–	659	706	727	–	686	700
100	200	666	–	660	697	721	–	682	711
100	1000	663	–	657	701	716	–	682	705
100	5000	663	658	658	700	715	680	680	705
300	20	699	–	700	742	715	–	706	767
300	50	706	–	700	725	717	–	701	759
300	200	713	–	704	723	719	–	706	746
300	1000	720	–	712	732	713	–	713	747
300	5000	723	714	714	733	713	713	713	747

Table 17: MDLM: valid per run. Columns are (mixture mode) $\times$ (arm); every cell is a single run of 1,024 sequences, with no averaging.

N	λ	marg. A	marg. A _{eu}	marg. B	marg. C	edlm. A	edlm. A _{eu}	edlm. B	edlm. C
10	20	893	–	873	913	900	–	894	935
10	50	885	–	871	908	894	–	886	935
10	200	886	–	880	910	890	–	887	922
10	1000	892	–	879	921	891	–	889	920
10	5000	894	878	878	922	889	888	888	925
30	20	920	–	922	922	936	–	943	954
30	50	924	–	923	932	929	–	933	929
30	200	920	–	916	920	930	–	923	933
30	1000	918	–	916	932	928	–	931	932
30	5000	916	916	915	939	923	932	932	935
100	20	934	–	930	944	952	–	960	972
100	50	921	–	918	937	932	–	947	955
100	200	910	–	921	942	916	–	921	932
100	1000	929	–	927	945	921	–	918	930
100	5000	932	926	926	941	920	917	917	934
300	20	929	–	928	941	963	–	969	978
300	50	917	–	902	935	957	–	950	958
300	200	925	–	917	920	948	–	937	944
300	1000	921	–	915	930	938	–	936	934
300	5000	923	911	911	937	936	934	934	924

Table 18: UDLM: valid per run. Columns are (mixture mode) $\times$ (arm); every cell is a single run of 1,024 sequences, with no averaging.

N	λ	marg. A	marg. A _{eu}	marg. B	marg. C	edlm. A	edlm. A _{eu}	edlm. B	edlm. C
10	20	253	–	255	260	244	–	259	271
10	50	262	–	259	268	247	–	259	291
10	200	263	–	249	272	248	–	267	292
10	1000	268	–	251	276	248	–	270	290
10	5000	268	253	253	274	252	273	273	287
30	20	275	–	284	312	270	–	271	258
30	50	283	–	284	308	277	–	281	281
30	200	297	–	292	328	297	–	282	309
30	1000	295	–	291	330	302	–	287	323
30	5000	292	284	284	325	304	288	288	314
100	20	287	–	294	313	305	–	310	308
100	50	286	–	294	332	314	–	330	327
100	200	305	–	324	355	332	–	335	350
100	1000	314	–	334	380	336	–	338	356
100	5000	315	331	331	381	335	338	338	357
300	20	313	–	316	337	301	–	321	377
300	50	319	–	314	355	330	–	350	409
300	200	381	–	373	393	356	–	371	428
300	1000	389	–	385	415	358	–	388	448
300	5000	396	393	393	419	362	391	391	452

Table 19: MDLM: novel per run. Columns are (mixture mode) $\times$ (arm); every cell is a single run of 1,024 sequences, with no averaging.

N	λ	marg. A	marg. A _{eu}	marg. B	marg. C	edlm. A	edlm. A _{eu}	edlm. B	edlm. C
10	20	239	–	240	228	222	–	252	250
10	50	236	–	231	223	228	–	242	241
10	200	232	–	241	212	240	–	253	229
10	1000	231	–	249	223	239	–	252	235
10	5000	236	248	248	223	240	250	250	240
30	20	246	–	259	287	252	–	264	282
30	50	238	–	258	248	236	–	283	272
30	200	245	–	272	222	254	–	282	255
30	1000	269	–	267	246	272	–	270	259
30	5000	267	260	260	242	265	271	271	255
100	20	288	–	267	285	300	–	287	280
100	50	259	–	232	271	263	–	295	265
100	200	279	–	257	282	307	–	300	289
100	1000	288	–	266	309	297	–	305	322
100	5000	287	261	261	309	300	308	308	327
300	20	251	–	269	270	327	–	277	306
300	50	259	–	245	281	290	–	292	306
300	200	316	–	302	324	306	–	331	356
300	1000	311	–	310	353	326	–	347	369
300	5000	307	305	305	342	325	336	336	363

Table 20: UDLM: novel per run. Columns are (mixture mode) $\times$ (arm); every cell is a single run of 1,024 sequences, with no averaging.

N	λ	marg. A	marg. A _{eu}	marg. B	marg. C	edlm. A	edlm. A _{eu}	edlm. B	edlm. C
10	20	0.4670	–	0.4730	0.4678	0.4730	–	0.4658	0.4738
10	50	0.4783	–	0.4771	0.4755	0.4744	–	0.4734	0.4805
10	200	0.4795	–	0.4815	0.4816	0.4787	–	0.4783	0.4841
10	1000	0.4816	–	0.4805	0.4832	0.4798	–	0.4792	0.4899
10	5000	0.4802	0.4813	0.4813	0.4834	0.4798	0.4790	0.4790	0.4894
30	20	0.4699	–	0.4699	0.4850	0.4780	–	0.4749	0.4787
30	50	0.4871	–	0.4817	0.4944	0.4870	–	0.4851	0.4965
30	200	0.4917	–	0.4860	0.5018	0.4890	–	0.4912	0.5049
30	1000	0.4905	–	0.4861	0.5007	0.4919	–	0.4913	0.5067
30	5000	0.4908	0.4873	0.4873	0.5002	0.4933	0.4907	0.4907	0.5077
100	20	0.4779	–	0.4719	0.4828	0.4830	–	0.4764	0.4845
100	50	0.4935	–	0.4855	0.4964	0.4953	–	0.4859	0.4997
100	200	0.4946	–	0.4958	0.5065	0.5010	–	0.4992	0.5125
100	1000	0.4993	–	0.4972	0.5078	0.5030	–	0.4988	0.5154
100	5000	0.5004	0.4968	0.4968	0.5086	0.5037	0.5007	0.5007	0.5136
300	20	0.4780	–	0.4807	0.4779	0.4813	–	0.4839	0.4858
300	50	0.4971	–	0.4975	0.4970	0.4946	–	0.4931	0.5031
300	200	0.5090	–	0.5091	0.5160	0.5010	–	0.4996	0.5151
300	1000	0.5124	–	0.5109	0.5194	0.5038	–	0.5008	0.5168
300	5000	0.5127	0.5118	0.5118	0.5186	0.5044	0.5021	0.5021	0.5164

Table 21: MDLM: prop per run. Columns are (mixture mode) $\times$ (arm); every cell is a single run of 1,024 sequences, with no averaging.

N	λ	marg. A	marg. A _{eu}	marg. B	marg. C	edlm. A	edlm. A _{eu}	edlm. B	edlm. C
10	20	0.4760	–	0.4798	0.4644	0.4818	–	0.4886	0.4869
10	50	0.4923	–	0.4893	0.4968	0.5014	–	0.5052	0.5024
10	200	0.5056	–	0.4980	0.5063	0.5121	–	0.5104	0.5079
10	1000	0.5023	–	0.4982	0.5144	0.5104	–	0.5106	0.5089
10	5000	0.5061	0.4977	0.4977	0.5124	0.5102	0.5111	0.5111	0.5091
30	20	0.4872	–	0.4840	0.4826	0.4882	–	0.4932	0.4917
30	50	0.5050	–	0.4987	0.5005	0.5076	–	0.5115	0.5168
30	200	0.5152	–	0.5149	0.5364	0.5211	–	0.5214	0.5367
30	1000	0.5224	–	0.5172	0.5360	0.5246	–	0.5221	0.5337
30	5000	0.5219	0.5176	0.5176	0.5373	0.5244	0.5243	0.5243	0.5307
100	20	0.4892	–	0.4834	0.4940	0.5019	–	0.5001	0.5071
100	50	0.5064	–	0.5091	0.5123	0.5194	–	0.5218	0.5236
100	200	0.5212	–	0.5244	0.5299	0.5330	–	0.5328	0.5392
100	1000	0.5286	–	0.5284	0.5415	0.5348	–	0.5337	0.5411
100	5000	0.5275	0.5287	0.5287	0.5448	0.5337	0.5341	0.5341	0.5385
300	20	0.4854	–	0.4928	0.4896	0.4937	–	0.4919	0.5078
300	50	0.5131	–	0.5100	0.5176	0.5297	–	0.5292	0.5309
300	200	0.5330	–	0.5325	0.5364	0.5426	–	0.5394	0.5516
300	1000	0.5333	–	0.5384	0.5490	0.5418	–	0.5414	0.5527
300	5000	0.5341	0.5378	0.5378	0.5479	0.5408	0.5384	0.5384	0.5538

Table 22: UDLM: prop per run. Columns are (mixture mode) $\times$ (arm); every cell is a single run of 1,024 sequences, with no averaging.

model	contrast	isolates	cells	Δ valid	Δ novel	Δ QED
MDLM	B – A _{eu}	restricting the target	8	+0.0 $\pm$ 0.0	+0.0 $\pm$ 0.0	-0.0000 $\pm$ 0.0000
UDLM	B – A _{eu}	restricting the target	8	-0.1 $\pm$ 0.1	+0.0 $\pm$ 0.0	+0.0000 $\pm$ 0.0000
MDLM	C – B	screening	40	+23.2 $\pm$ 2.3*	+27.3 $\pm$ 2.8*	+0.0092 $\pm$ 0.0009*
UDLM	C – B	screening	40	+17.7 $\pm$ 2.3*	+3.6 $\pm$ 4.0	+0.0070 $\pm$ 0.0012*
MDLM	C – A	<i>end-to-end</i>	40	+11.9 $\pm$ 3.5*	+32.0 $\pm$ 4.0*	+0.0073 $\pm$ 0.0008*
UDLM	C – A	<i>end-to-end</i>	40	+14.8 $\pm$ 2.0*	+7.7 $\pm$ 3.5*	+0.0066 $\pm$ 0.0011*

* marks $|t| > 2.1$. These are paired across grid cells, not across seeds: this sweep has one seed per cell, so the spread mixes cell-to-cell variation with run-to-run noise and the $\pm$ is not a pure standard error.

Table 23: Paired within each (N, λ) and mixture mode, which removes both and leaves only the contrast named.

model	contrast	isolates	cells	≥ 0.5	≥ 0.55	≥ 0.6	≥ 0.65	≥ 0.7
MDLM	B – A _{eu}	restricting the target	8	+0.0 $\pm$ 0.0	+0.0 $\pm$ 0.0	+0.0 $\pm$ 0.0	+0.0 $\pm$ 0.0	+0.0 $\pm$ 0.0
UDLM	B – A _{eu}	restricting the target	8	+0.0 $\pm$ 0.0	+0.0 $\pm$ 0.0	+0.0 $\pm$ 0.0	+0.0 $\pm$ 0.0	+0.0 $\pm$ 0.0
MDLM	C – B	screening	40	+29.6 $\pm$ 3.4*	+18.6 $\pm$ 1.7*	+8.9 $\pm$ 1.1*	+1.6 $\pm$ 0.3*	-0.0 $\pm$ 0.0
UDLM	C – B	screening	40	+8.6 $\pm$ 3.0*	+14.9 $\pm$ 3.1*	+11.7 $\pm$ 1.8*	+1.2 $\pm$ 0.5*	-0.0 $\pm$ 0.1
MDLM	C – A	<i>end-to-end</i>	40	+30.1 $\pm$ 3.4*	+19.1 $\pm$ 1.9*	+7.4 $\pm$ 1.0*	+2.0 $\pm$ 0.4*	+0.0 $\pm$ 0.0
UDLM	C – A	<i>end-to-end</i>	40	+11.4 $\pm$ 2.8*	+16.1 $\pm$ 2.8*	+12.3 $\pm$ 2.2*	+1.7 $\pm$ 0.7*	+0.1 $\pm$ 0.1

Δ hits: the change in the number of novel molecules clearing each QED bar, out of 1,024 generated. The high bars count in the single digits per run, so those columns are noise-dominated and read as unresolved rather than null.

Table 24: The same paired contrast read at every quality bar. A conclusion that holds at one bar and reverses at another is a property of the bar, not of the method, so all five are shown.

contrast	mode	model	cells	≥ 0.5	≥ 0.55	≥ 0.6	≥ 0.65	≥ 0.7
screening	marginal	MDLM	20	+26.1 $\pm$ 4.1*	+16.0 $\pm$ 2.3*	+9.9 $\pm$ 1.6*	+2.2 $\pm$ 0.6*	+0.1 $\pm$ 0.0
screening	marginal	UDLM	20	+12.1 $\pm$ 4.3*	+19.6 $\pm$ 3.8*	+11.8 $\pm$ 2.8*	+0.3 $\pm$ 0.8	-0.3 $\pm$ 0.2
screening	edlm	MDLM	20	+33.0 $\pm$ 5.5*	+21.1 $\pm$ 2.5*	+7.8 $\pm$ 1.4*	+1.1 $\pm$ 0.2*	-0.1 $\pm$ 0.1
screening	edlm	UDLM	20	+5.2 $\pm$ 4.3	+10.3 $\pm$ 4.7*	+11.7 $\pm$ 2.3*	+2.1 $\pm$ 0.8*	+0.2 $\pm$ 0.1
end-to-end	marginal	MDLM	20	+26.8 $\pm$ 3.5*	+15.4 $\pm$ 2.1*	+8.2 $\pm$ 1.4*	+1.8 $\pm$ 0.7*	+0.1 $\pm$ 0.0
end-to-end	marginal	UDLM	20	+11.8 $\pm$ 3.5*	+18.1 $\pm$ 4.0*	+12.7 $\pm$ 3.0*	+0.6 $\pm$ 0.8	-0.2 $\pm$ 0.1
end-to-end	edlm	MDLM	20	+33.5 $\pm$ 5.9*	+22.6 $\pm$ 2.9*	+6.7 $\pm$ 1.4*	+2.1 $\pm$ 0.3*	+0.0 $\pm$ 0.0
end-to-end	edlm	UDLM	20	+10.9 $\pm$ 4.5*	+14.1 $\pm$ 4.0*	+12.0 $\pm$ 3.1*	+2.8 $\pm$ 1.2*	+0.3 $\pm$ 0.1*

20 cells per row against 40 in the pooled table, so the power is halved and the outer bars lose significance in some combinations while keeping their sign. The one place the modes differ in kind is the 0.50 bar on UDLM: +12.1 under **marginal** against +5.2 (n.s.) under **edlm**.

Table 25: Δ hits split by mixture mode. Screening changes the *proposal*, which is upstream of the choice of sampler, so it should help in both — and it does, at the 0.55 and 0.60 bars in all four (mode, model) combinations.

model	cells	Δ valid	Δ novel	Δ QED	Δ $h_{\geq 0.5}$	Δ $h_{\geq 0.55}$	Δ $h_{\geq 0.6}$	Δ $h_{\geq 0.65}$	Δ $h_{\geq 0.7}$
MDLM	8	-11.1 ± 6.3	$+3.4 \pm 6.0$	$-0.0019 \pm 0.0006^*$	-0.1 ± 3.6	$+1.4 \pm 2.4$	$-1.9 \pm 0.7^*$	$+0.2 \pm 0.6$	$+0.1 \pm 0.1$
UDLM	8	-3.9 ± 2.7	$+1.5 \pm 4.6$	-0.0011 ± 0.0013	$+0.4 \pm 4.1$	$+2.6 \pm 2.8$	$+0.4 \pm 1.6$	$+0.0 \pm 1.0$	$+0.2 \pm 0.2$

The substitution replaces a Monte Carlo estimate of the base kernel with its closed form wherever the N weights tie. For absorbing state the two have the *same mean*, since the posterior is linear in x_0 — so this removes variance, not bias, and an equal mean does not imply an equal metric. UDLM should move least: its uniform-weight rate is 0.156 against MDLM’s 0.359. Whichever way it comes out, `ours.yaml` calling it an improvement “at no cost” claims more than a variance argument can support.

Table 26: $A_{\text{eu}} - A$: what the exact-uniform step does on its own, paired at matched (N, mode) at $\lambda = 5000$. The only difference between the two is the flag.

model	λ	≥ 0.5	≥ 0.55	≥ 0.6	≥ 0.65	≥ 0.7
MDLM	20	+13.5	+10.9	+4.9	+0.2	+0.0
MDLM	50	+22.0	+16.8	+8.0	+0.6	+0.0
MDLM	200	+32.9	+20.6	+10.2	+1.9	+0.0
MDLM	1000	+41.1	+23.0	+10.6	+2.9	+0.0
MDLM	5000	+38.5	+21.6	+10.8	+2.6	-0.1
UDLM	20	+8.2	+8.2	+1.9	+0.0	+0.0
UDLM	50	+2.9	+7.0	+6.6	-0.2	+0.2
UDLM	200	+2.2	+14.8	+13.0	+0.6	+0.1
UDLM	1000	+14.5	+22.8	+18.5	+2.9	-0.1
UDLM	5000	+15.2	+22.0	+18.6	+2.8	-0.4

Each row averages 8 cells ($N \in \{10, 30, 100, 300\} \times 2$ mixture modes).

Table 27: Screening gain (C – B) against tilt strength, averaged over N and mixture mode. The gain grows with λ on both models, opposite to the direction predicted for `exclude_invalid`.

But the two knobs do different things, which is why the comparison is not a wash. Raising N adds hits at *every* bar and most at the low ones; screening moves probability mass *up*, gaining at 0.60–0.65 and losing at 0.50. If the deliverable is "molecules above a demanding threshold" the two are not interchangeable at any budget.

The cost penalty is ours, not the method’s. The reward-call model predicts screening should cost about $1.2\times$, not $5\times$: parsing is 0.042 ms against 0.811 ms for parse+QED, and only N candidates ever reach QED. Checking the gap arithmetically at $N = 300$ on MDLM — 39.3 M extra candidates over a measured 786 s, i.e. $20\mu\text{s}$ each — parsing accounts for just 52 s of it once spread over 32 workers. The remaining ~ 734 s is *serial decoding*, turning candidate token tensors into SMILES strings in Python. That is the same bottleneck the shelved `fast_decode` attempt failed to remove. So the honest statement is: screening is a clear win per reward call and a loss per second *in this implementation*, and the cost verdict would flip with a decoder that is not Python string joins. That is a falsifiable claim about our code, not a property of the estimator.

arm	N	cost (s)	≥ 0.5	≥ 0.55	≥ 0.6	≥ 0.65	≥ 0.7
B	10	46	104	44	13	1	0
B	30	50	124	57	16	3	0
C	10	58	116	54	19	2	0
B	100	68	168	80	30	6	0
C	30	83	175	83	36	7	0
B	300	121	221	115	37	7	1
C	100	320	208	114	42	10	0
C	300	907	264	135	55	9	1

Read down the cost column and compare rows of similar cost. **B** at $N = 300$ costs 121 s (MDLM) / 143 s (UDLM); **C** at $N = 100$ costs 320 s / 371 s, i.e. $2.6\times$ more, and does not clearly beat it.

Table 28: MDLM: hits against measured cost, ordered by cost. Each row takes the best hits over λ and mixture mode at that (arm, N), i.e. it assumes the tilt is tuned. Cost is the mean wall clock of the 8 cells behind the row.

arm	N	cost (s)	≥ 0.5	≥ 0.55	≥ 0.6	≥ 0.65	≥ 0.7
B	10	52	154	78	15	2	0
B	30	56	193	104	31	6	0
C	10	64	148	76	21	2	0
B	100	77	224	131	49	10	0
C	30	126	184	115	42	8	1
B	300	143	266	156	67	14	3
C	100	371	233	154	74	16	0
C	300	898	293	202	98	24	2

Read down the cost column and compare rows of similar cost. **B** at $N = 300$ costs 121 s (MDLM) / 143 s (UDLM); **C** at $N = 100$ costs 320 s / 371 s, i.e. $2.6\times$ more, and does not clearly beat it.

Table 29: UDLM: hits against measured cost, ordered by cost. Each row takes the best hits over λ and mixture mode at that (arm, N), i.e. it assumes the tilt is tuned. Cost is the mean wall clock of the 8 cells behind the row.

5.1 A cost model, and why QED is the worst case

Split the per-candidate cost into the part that scales with the oversampling factor k and the part that does not. Every one of the kN candidates must be decoded and validity-checked, at cost $c = d + v$; only the N survivors reach the reward, at cost r . Then

$$\text{cost}_B = N(c + r), \quad \text{cost}_C = N(kc + r), \quad \rho = \frac{\text{cost}_C}{\text{cost}_B} = \frac{k + \theta}{1 + \theta}, \quad \theta \equiv r/c.$$

$\rho \rightarrow 1$ as $\theta \rightarrow \infty$: **screening is asymptotically free in the reward cost.** Requiring $\rho \leq 1.2$ gives $\theta \geq 5(k - 1.2)$, i.e. $\theta \geq 4, 9, 19, 44$ for $k = 2, 3, 5, 10$.

Where QM9 sits. $c \approx 20 \mu\text{s}$, dominated by serial decoding, and $r \approx 0.811 \text{ ms}/32 = 25 \mu\text{s}$ once the RDKit calls are spread over the worker pool — so $\theta \approx 1.25$ and $\rho \approx 2.8$, against 4.7 and 7.5 measured at $N = 100$ and 300. **QED is close to the worst case this method can be run in.** Not because QED is cheap in absolute terms — 0.811 ms is 40× our decode — but because we parallelised the expensive part 32-fold and left the cheap part serial, which collapses θ to about 1.

A neural reward moves θ the right way, and by a lot. At $r = 500 \mu\text{s}$ effective, $\rho = 1.15$; at $r = 5 \text{ ms}$, $\rho = 1.02$. A GPU property predictor or a preference model is also *harder* to parallelise 32-fold than 32 independent RDKit calls — it is one batched GPU op, not an embarrassingly parallel CPU pool — so its effective r stays high. Both effects push in the same direction. **The regime where screening is cheapest is exactly the regime where reward models are actually used**, which means the QM9 cost result is a floor on the method rather than a verdict on it.

One caveat on the language plan. The decomposition needs the feasibility check to be cheap *relative to* the quality reward. Parse against QED is a structural gap: 0.042 vs 0.811 ms, different kinds of computation. A safety classifier against a preference model is not — both are neural forward passes, and the gap is only the size ratio of the two networks. That makes it a quantitative claim to be measured, not one that can be asserted from the shape of the pipeline.

$\theta = r/c$	ρ at $k=5$	regime
0.3	4.08	reward as cheap as decoding
1.25	2.78	QM9 + QED, 32 workers (ours)
3	2.00	unparallelised RDKit
10	1.36	small GPU predictor
19	1.20	break-even target for $k = 5$
100	1.04	preference model / LLM judge

$c = d + v$ is decode plus validity check, paid on all kN candidates; r is the reward, paid on N . Both are effective costs, i.e. after whatever parallelism each enjoys.

Table 30: Screening overhead ρ against the reward-to-scan cost ratio θ . The rows are the model evaluated at illustrative θ ; only the QM9 row is measured.

Note for anyone reusing this. Screening is incompatible with `position.selection=cv.conf`: that control variate needs $\mathbb{E}[\bar{x}_0^{(0)}] = x_\theta$ exactly, which holds only when the candidates are drawn from x_θ itself.

Two earlier versions of this experiment were discarded rather than reported. The first padded short pools with invalid candidates while they still carried weight e^0 , which makes the proposal depend on how many happened to be valid and is not a correct estimator of anything; the padding fired on 53 % of MDLM and 79 % of UDLM sequences, so this was the dominant path, not a corner. Its numerical effect was probably small — at $\lambda \geq 50$ the exclusion is a no-op, verified byte-identical at $\lambda = 200$, because $e^{\lambda \cdot 0.5}$ against e^0 already suppresses invalid candidates — but the justification did not hold, so it was rerun.